

JAVA LAMBDA EXPRESSIONS & FUNCTIONAL INTERFACES

DETAILED THEORY + PRACTICAL IMPLEMENTATION + ANONYMOUS CLASSES + BUILT-IN FUNCTIONAL INTERFACES + COLLECTIONS + STREAMS + EXERCISES + INTERVIEW PREPARATION

For Java 8, 11, 17 and 21 | Beginner-to-intermediate study notes

How to Study This Chapter

Do not memorize lambda syntax first. Understand the problem that lambda solves. Java often needs to pass a small piece of behavior to another method: a condition, a calculation, a sorting rule, a printing action, or a transformation. Traditionally this required a class or anonymous class. Functional interfaces provide a standard type for such behavior, and lambda expressions provide a compact implementation.

Recommended sequence: Anonymous class → Functional interface → Lambda → Built-in interfaces → Collections → Method references → Streams → Exercises.

1. Why Lambda Expressions Were Introduced

Consider a simple requirement: “Give me a piece of code that can add two numbers.” In traditional Java, we could create a class or interface implementation. The implementation can become verbose when the behavior is tiny.

Traditional approach:

```
interface Calculator {
    int calculate(int a, int b);
}

class Addition implements Calculator {
    @Override
    public int calculate(int a, int b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Addition();
        System.out.println(c.calculate(10, 20));
    }
}
```

The actual behavior is only **a + b**, but several lines are required. Java 8 introduced lambda expressions so this can be expressed directly:

```
Calculator c = (a, b) -> a + b;
System.out.println(c.calculate(10, 20));
```

This is the core motivation: **represent behavior as data-like values that can be passed around.**

2. What Exactly Is a Lambda Expression?

Definition: A lambda expression is an anonymous function-like expression that provides an implementation of the single abstract method of a functional interface.

A lambda has three important parts:

- parameters — the input values
- -> — lambda operator
- body — the operation to perform

```
(parameters) -> expression

(parameters) -> {
    statements;
}
```

Examples:

```
() -> System.out.println("Hello");

x -> x * x;

(a, b) -> a + b;

name -> {
    String message = "Hello " + name;
    System.out.println(message);
};
```

Important: A lambda does not define an ordinary named Java method. Its type is determined by the target functional interface.

3. Lambda Syntax — Every Common Form

Situation	Syntax	Example
No parameter	() -> body	() -> System.out.println("Hi")
One parameter	x -> body	x -> x * 2
One parameter with type	(int x) -> body	(int x) -> x * 2
Two parameters	(a,b) -> body	(a,b) -> a+b
Expression	x -> expression	x -> x+10
Block	x -> { ... }	x -> { return x+10; }

For one parameter, parentheses are optional when the type is omitted. If you explicitly write the parameter type, follow Java's lambda syntax rules and use parentheses.

4. Expression Body vs Block Body

An expression lambda is the shortest form and automatically produces the expression's value when a value is expected.

```
Function square = x -> x * x;

System.out.println(square.apply(5)); // 25
```

A block lambda uses braces and is useful when multiple statements are required.

```
Function square = x -> {
    int result = x * x;
    System.out.println("Calculating...");
    return result;
};
```

Common mistake:

```
Function f = x -> {
    x * x; // ERROR: no return
```

```
};
```

Correct:

```
Function f = x -> {  
    return x * x;  
};
```

5. Functional Interface — Deep Explanation

Definition: A functional interface is an interface with exactly one abstract method. It can contain any number of default and static methods because those methods already have implementations.

```
@FunctionalInterface  
interface Calculator {  
    int calculate(int a, int b);  
}
```

Why only one abstract method? Because Java must know exactly which method the lambda is implementing. If two abstract methods existed, the compiler could not map one lambda body unambiguously.

```
interface Wrong {  
    void first();  
    void second();  
}  
  
// Wrong x = () -> System.out.println("Hi");  
// ERROR: Wrong is not a functional interface
```

Functional Interface $\neq$ interface with only one method total. It means one *abstract* method. Default and static methods do not count toward the single-abstract-method requirement.

6. @FunctionalInterface Annotation

The annotation is not mandatory, but it is strongly recommended when you intentionally create a functional interface.

```
@FunctionalInterface  
interface Greeting {  
    void greet(String name);  
}
```

If someone later adds another abstract method:

```
@FunctionalInterface  
interface Greeting {  
    void greet(String name);  
    void print(); // compilation error  
}
```

The annotation therefore acts as a compile-time safety check and documents the design intention.

7. Functional Interface with Default and Static Methods

```
@FunctionalInterface  
interface EmployeeAction {  
  
    void execute(String name);  
  
    default void start() {  
        System.out.println("Starting...");  
    }  
  
    static void info() {
```

```
System.out.println("Employee action interface");
}
}
```

This is valid because only `execute(String)` is abstract.

8. Anonymous Class — Before Understanding Lambda

Definition: An anonymous class is a class without a declared name that is instantiated at the point where it is needed.

```
interface Greeting {
    void greet(String name);
}

Greeting g = new Greeting() {
    @Override
    public void greet(String name) {
        System.out.println("Hello " + name);
    }
};

g.greet("Ujjwal");
```

This was a common way to provide one-time behavior before lambda expressions.

The same functional-interface implementation using a lambda:

```
Greeting g = name -> System.out.println("Hello " + name);

g.greet("Ujjwal");
```

9. Anonymous Class vs Lambda — Deep Difference

Feature	Anonymous Class	Lambda
What it represents	Anonymous class instance	Behavior for a functional interface
Target	Class/interface implementation	Functional interface
Abstract methods	Can implement multiple abstract methods if interface	Target must have exactly one abstract method
this	Refers to anonymous class instance	Refers to enclosing instance
super	Can refer to superclass/interface behavior in class	Does not create a new this context
Syntax	Verbose	Compact
State	Can declare instance fields/methods	Does not declare a class body
Best for	One-off customized class behavior	Small behavior/callback

10. The Most Important Concept: Target Type

A lambda by itself does not tell Java whether it returns a number, string, boolean, or nothing. The surrounding target type provides that information.

```
Predicate p = x -> x > 10;
// lambda means: Integer -> boolean

Function f = x -> "Number: " + x;
// lambda means: Integer -> String

Consumer c = x -> System.out.println(x);
// lambda means: String -> void
```

This is called **target typing**. The compiler uses the expected functional-interface type to infer the parameter and return information.

11. Built-in Functional Interfaces — `java.util.function`

Java provides many ready-made functional interfaces so you do not need to create your own interface for common patterns.

Interface	Input	Output	Method	Meaning
<code>Predicate<T></code>	<code>T</code>	<code>boolean</code>	<code>test(T)</code>	Check condition
<code>Function<T,R></code>	<code>T</code>	<code>R</code>	<code>apply(T)</code>	Transform value
<code>Consumer<T></code>	<code>T</code>	<code>void</code>	<code>accept(T)</code>	Perform action
<code>Supplier<T></code>	<code>none</code>	<code>T</code>	<code>get()</code>	Produce value
<code>UnaryOperator<T></code>	<code>T</code>	<code>T</code>	<code>apply(T)</code>	Transform same type
<code>BinaryOperator<T></code>	<code>T,T</code>	<code>T</code>	<code>apply(T,T)</code>	Combine same type
<code>BiPredicate<T,U></code>	<code>T,U</code>	<code>boolean</code>	<code>test(T,U)</code>	Two-input condition
<code>BiFunction<T,U,R></code>	<code>T,U</code>	<code>R</code>	<code>apply(T,U)</code>	Two-input transformation
<code>BiConsumer<T,U></code>	<code>T,U</code>	<code>void</code>	<code>accept(T,U)</code>	Two-input action

12. Predicate — Detailed Practical Implementation

Predicate represents a condition. Its abstract method is `boolean test(T t)`.

```
import java.util.function.Predicate;

public class PredicateDemo {
    public static void main(String[] args) {

        Predicate even =
            number -> number % 2 == 0;

        System.out.println(even.test(10)); // true
        System.out.println(even.test(7)); // false
    }
}
```

Real-world example: validating employees.

```
Predicate highSalary =
    employee -> employee.getSalary() > 50000;

if (highSalary.test(employee)) {
    System.out.println("High salary employee");
}
```

Predicate composition:

```
Predicate even = n -> n % 2 == 0;
Predicate greaterThan10 = n -> n > 10;

Predicate rule =
    even.and(greaterThan10);

System.out.println(rule.test(20)); // true
System.out.println(rule.test(8)); // false
```

Useful methods include `and()`, `or()`, and `negate()`.

13. Function — Detailed Practical Implementation

Function accepts one type and returns another type.

```
import java.util.function.Function;

Function length =
    text -> text.length();

System.out.println(length.apply("Java")); // 4
```

Chaining functions:

```
Function trim =
    text -> text.trim();

Function upper =
    text -> text.toUpperCase();

Function pipeline =
    trim.andThen(upper);

System.out.println(pipeline.apply(" java "));
// JAVA
```

Real-world use: converting an `Employee` into a display name, converting a `DTO` into an entity, or transforming raw input into another representation.

14. Consumer — Detailed Practical Implementation

Consumer accepts a value and returns nothing.

```
import java.util.function.Consumer;

Consumer print =
text -> System.out.println(text);

print.accept("Hello Java");
```

A Consumer is useful for logging, printing, sending data, updating an object, or performing an action.

```
Consumer log =
message -> System.out.println("LOG: " + message);

log.accept("Server started");
```

15. Supplier — Detailed Practical Implementation

Supplier takes no input and supplies a value.

```
import java.util.function.Supplier;

Supplier welcome =
() -> "Welcome to Java";

System.out.println(welcome.get());

Supplier randomNumber =
() -> (int)(Math.random() * 100);

System.out.println(randomNumber.get());
```

Supplier is useful when value creation should be delayed until get() is called.

16. UnaryOperator and BinaryOperator

```
UnaryOperator doubleValue =
x -> x * 2;

System.out.println(doubleValue.apply(5)); // 10

BinaryOperator add =
(a, b) -> a + b;

System.out.println(add.apply(10, 20)); // 30
```

UnaryOperator is essentially Function. BinaryOperator is essentially BiFunction.

17. Primitive Functional Interfaces

For numeric work, Java also provides specialized interfaces such as IntPredicate, IntFunction, IntConsumer, IntSupplier, ToIntFunction, and related Long/Double variants. They can reduce boxing and unboxing overhead in appropriate cases.

```
IntPredicate positive = x -> x > 0;
System.out.println(positive.test(5));
```

Do not use primitive specializations automatically everywhere. Use them when they fit the API and workload.

18. Lambda with Collections — forEach

```
import java.util.*;
```

```
List names =
Arrays.asList("Amit", "Rahul", "Sneha");

names.forEach(name ->
System.out.println(name));
```

Equivalent anonymous-class style is much more verbose. Lambda makes the operation readable.

19. Lambda with Comparator — Sorting

```
List names =
new ArrayList<>(List.of(
"Ujjwal", "Amit", "Rahul", "Neha"
));

names.sort((a, b) -> a.compareTo(b));

System.out.println(names);
```

Reverse order:

```
names.sort((a, b) -> b.compareTo(a));
```

Using a comparator is one of the most important practical lambda use cases.

20. Employee Sorting — Realistic Example

```
employees.sort(
(e1, e2) -> Double.compare(
e1.getSalary(),
e2.getSalary()
)
);
```

Sort by name:

```
employees.sort(
(e1, e2) -> e1.getName().compareTo(e2.getName())
);
```

Modern Java also provides `Comparator.comparing(Employee::getName)`, which combines lambda concepts with method references.

21. Method Reference — Lambda Simplification

A method reference is a compact form of a lambda when an existing method already performs exactly the required action.

```
names.forEach(name ->
System.out.println(name));

names.forEach(System.out::println);
```

Common forms:

```
ClassName::staticMethod
object::instanceMethod
ClassName::instanceMethod
ClassName::new
```

```
Function f =
String::length;
```

```
Supplier> s =
ArrayList::new;
```

22. Lambda with Streams — filter and map

Lambdas are heavily used by the Stream API. Example:

```
List numbers =  
List.of(10, 15, 20, 25, 30);  
  
numbers.stream()  
.filter(n -> n % 2 == 0)  
.forEach(n -> System.out.println(n));
```

Here, filter receives a Predicate-like condition, while forEach receives a Consumer-like action.

```
List names =  
List.of("amit", "rahul", "sneha");  
  
List result =  
names.stream()  
.map(name -> name.toUpperCase())  
.toList();  
  
System.out.println(result);
```

map transforms values; filter selects values; forEach performs an action.

23. Passing Lambda as a Method Argument

This is where lambdas become especially powerful. A method can accept behavior as a parameter.

```
static void process(  
int value,  
Predicate rule) {  
  
    if (rule.test(value)) {  
        System.out.println("Accepted");  
    } else {  
        System.out.println("Rejected");  
    }  
}  
  
public static void main(String[] args) {  
    process(20, x -> x > 10);  
    process(5, x -> x > 10);  
}
```

The method does not know what the condition is. The caller decides by supplying a lambda.

24. Returning a Lambda from a Method

```
static Predicate greaterThan(int limit) {  
    return number -> number > limit;  
}  
  
Predicate rule = greaterThan(50);  
  
System.out.println(rule.test(75)); // true  
System.out.println(rule.test(20)); // false
```

This demonstrates that functional-interface objects can be returned from methods too.

25. Variable Capture and Effectively Final

A lambda can capture a local variable when that local variable is final or effectively final.

```
int limit = 50;
```

```
Predicate p =
number -> number > limit;

System.out.println(p.test(75)); // true
```

The following is not allowed because limit is modified:

```
int limit = 50;

Predicate p =
number -> number > limit;

limit = 100; // invalidates effective-final rule
```

The idea is important for interviews: **local variables captured by a lambda must be final or effectively final.**

26. Lambda and this — Important Interview Concept

A lambda does not introduce a new this reference. Inside a lambda, this refers to the enclosing object.

```
class Demo {
private String name = "Outer";

void test() {
Runnable r = () ->
System.out.println(this.name);

r.run();
}
}
```

This differs from an anonymous class, where this refers to the anonymous class instance.

27. Exception Handling in Lambdas

Checked exceptions follow the abstract method's declared throws clause. A lambda cannot simply throw a checked exception through a functional interface whose method does not declare that exception.

```
@FunctionalInterface
interface Task {
void run() throws Exception;
}

Task t = () -> {
throw new Exception("Problem");
};
```

For standard interfaces such as Runnable, whose run() does not declare checked exceptions, checked exceptions need to be handled inside the lambda or wrapped appropriately.

28. Generic Functional Interface — Custom Example

```
@FunctionalInterface
interface Converter {
R convert(T value);
}

public class Main {
public static void main(String[] args) {

Converter converter =
text -> Integer.parseInt(text);

System.out.println(
```

```
converter.convert("123")
);
}
}
```

This demonstrates that a custom functional interface can itself be generic.

29. Complete Practical Mini Project — Employee Management

This example combines a custom functional interface, Predicate, Function, Consumer, sorting, filtering and method references.

```
import java.util.*;
import java.util.function.*;

class Employee {
    private int id;
    private String name;
    private String department;
    private double salary;

    Employee(int id, String name,
        String department, double salary) {
        this.id = id;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }

    @Override
    public String toString() {
        return id + " | " + name + " | "
            + department + " | " + salary;
    }
}

public class EmployeeManager {

    static void print(
        List employees,
        Predicate condition) {

        for (Employee e : employees) {
            if (condition.test(e)) {
                System.out.println(e);
            }
        }
    }

    public static void main(String[] args) {

        List employees = new ArrayList<>(
            List.of(
                new Employee(1, "Amit", "IT", 60000),
                new Employee(2, "Sneha", "HR", 45000),
                new Employee(3, "Rahul", "IT", 80000),
                new Employee(4, "Neha", "Sales", 55000)
            )
        );

        // Predicate
        Predicate itEmployee =
            e -> e.getDepartment().equals("IT");

        print(employees, itEmployee);

        // Salary condition
        print(employees,
            e -> e.getSalary() > 50000);
    }
}
```

```

// Function
Function employeeName =
Employee::getName;

System.out.println(
employeeName.apply(employees.get(0))
);

// Consumer
Consumer printer =
System.out::println;

employees.forEach(printer);

// Sorting
employees.sort(
Comparator.comparing(Employee::getName)
);

System.out.println("Sorted:");
employees.forEach(System.out::println);
}
}

```

What to understand from this project:

- Predicate represents a condition.
- Function transforms an Employee into another value.
- Consumer performs an action.
- Comparator controls sorting.
- Method references make simple lambdas shorter.
- The print method accepts behavior rather than hard-coding one condition.

30. Common Mistakes and Their Corrections

Mistake: Trying to write a lambda without a target type. **Fix:** Assign it to or pass it where a functional interface is expected.

Mistake: Creating a functional interface with two abstract methods. **Fix:** Keep exactly one abstract method.

Mistake: Forgetting return in a block lambda. **Fix:** Use return when the functional method returns a value.

Mistake: Modifying a captured local variable. **Fix:** Make the variable final/effectively final or redesign the code.

Mistake: Thinking Predicate returns an object. **Fix:** Predicate always represents a boolean condition.

Mistake: Confusing Consumer and Supplier. **Fix:** Consumer receives; Supplier produces.

Mistake: Assuming lambda and anonymous class are exactly the same. **Fix:** They have different semantics, especially around this and class behavior.

31. When Should You Use a Lambda?

- Use lambdas for short, focused behavior.
- Use Predicate for reusable conditions.
- Use Function for transformations.

- Use Consumer for actions.
- Use Supplier for deferred value creation.
- Use Comparator lambdas for custom sorting.
- Use method references when they improve readability.
- Use a named method/class when logic becomes large or needs meaningful identity, state, or reuse beyond a small behavior.

32. When NOT to Force a Lambda

A lambda is not automatically better just because it is shorter. If the body becomes large, contains complicated branching, needs several helper methods or maintains substantial state, a named method/class may be easier to understand and test.

Good code is not “maximum lambda usage.” Good code is **clear behavior expressed with an appropriate abstraction.**

33. Exercises — Basic to Advanced

1. **[Beginner]** Create Greeting with greet(String) and implement it using a lambda.
2. **[Beginner]** Create Calculator with one abstract method and create lambdas for addition, subtraction, multiplication and division.
3. **[Beginner]** Create Predicate for even numbers.
4. **[Beginner]** Create Function to calculate string length.
5. **[Beginner]** Create Consumer to print text in uppercase.
6. **[Beginner]** Create Supplier that generates a random number.
7. **[Intermediate]** Create Predicate and combine conditions using and(), or(), and negate().
8. **[Intermediate]** Create a List and print all numbers greater than 50 using a lambda.
9. **[Intermediate]** Sort a list of Employee objects by salary using Comparator and lambda.
10. **[Intermediate]** Sort Employee objects by department and then name.
11. **[Intermediate]** Write a method that accepts Predicate and prints matching employees.
12. **[Intermediate]** Write a method that accepts Function and prints transformed employee data.
13. **[Advanced]** Return Predicate from a method based on a supplied threshold.
14. **[Advanced]** Build an Employee filtering utility using Predicate composition.
15. **[Advanced]** Rewrite simple lambdas as method references where possible.
16. **[Advanced]** Create a Stream pipeline using filter, map, sorted and forEach.
17. **[Interview]** Explain why an interface with one abstract method can be implemented by a lambda.
18. **[Interview]** Explain target typing and effectively final variables with examples.
19. **[Interview]** Explain the difference between lambda this and anonymous-class this.

34. Interview Questions with Answer Direction

Q: What is lambda expression?

Answer: Anonymous function-like syntax used to provide behavior for a functional interface.

Q: What is functional interface?

Answer: Interface with exactly one abstract method; it may contain default/static methods.

Q: What is @FunctionalInterface?

Answer: Compiler-checked annotation documenting and enforcing the single-abstract-method design.

Q: Can lambda implement a normal interface with two abstract methods?

Answer: No.

Q: Why is lambda shorter than anonymous class?

Answer: Target type supplies the interface and method information, so class/method boilerplate is omitted.

Q: Difference between Predicate and Function?

Answer: Predicate returns boolean; Function transforms T into R.

Q: Difference between Consumer and Supplier?

Answer: Consumer accepts input and returns void; Supplier takes no input and returns a value.

Q: What is effectively final?

Answer: A local variable that is not declared final but is assigned only once.

Q: What is method reference?

Answer: Compact syntax referring to an existing method when its signature fits the target functional interface.

Q: Does lambda create a new this?

Answer: No; lambda this refers to the enclosing instance.

Q: Can functional interface have default methods?

Answer: Yes.

Q: Can functional interface have static methods?

Answer: Yes.

Q: Can a functional interface have Object methods?

Answer: Methods that match public methods of Object do not count as abstract methods for the functional-interface determination.

35. Quick Revision Sheet

Lambda: (parameters) -> body

Functional Interface: exactly one abstract method

Predicate: $T \rightarrow \text{boolean}$

Function: $T \rightarrow R$

Consumer: $T \rightarrow \text{void}$

Supplier: $() \rightarrow T$

UnaryOperator: $T \rightarrow T$

BinaryOperator: $(T, T) \rightarrow T$

Method Reference: existing method used as behavior, e.g. `System.out::println`

Captured local variable: final/effectively final

Core idea: Functional interface defines the required behavior; lambda supplies the implementation.

36. Final Practical Challenge

Build a small Employee Management program with at least 8 employees. Your program must:

- Print all employees using Consumer.
- Find employees from a selected department using Predicate.
- Find employees above a selected salary.
- Sort employees alphabetically using Comparator.
- Sort employees by salary.
- Convert Employee objects into names using Function.
- Use at least one method reference.
- Use at least one custom `@FunctionalInterface`.
- Add a menu or separate methods so each operation can be tested independently.

Do not copy the complete mini-project above. Try implementing this challenge yourself first. Then compare your design with the concepts in the chapter.

37. Final Takeaway

The most important relationship is:

```
Functional Interface
↓
defines one abstract behavior
↓
Lambda Expression
↓
provides the implementation
↓
Method / Collection / Stream
↓
uses that behavior
```

Once you understand this flow, interfaces such as Predicate, Function, Consumer and Supplier stop looking like separate topics. They become ready-made functional interfaces that allow Java APIs to accept behavior cleanly.